AC Lecture 1

Intro + Turing Machines

The Complete Guide to "What is a Problem?" and "What is an Algorithm?"

Algorithms and Computability — Semester 6
Cheatsheet for split-view study with slides

Contents

1 The Big Picture — Why This Course Exists

Before We Start — Why Should You Care About This Course?

Imagine you're a software developer and your boss asks: "Write me a program that checks whether any given program will crash." You spend weeks, months — and you can't do it. Is it because you're bad at programming? **No.** It's because it's **mathematically impossible**. No one can do it. Not now, not ever.

But how do you *prove* something is impossible? You can't just say "I tried really hard and failed." You need a **formal definition** of what a "program" and a "problem" even *are*. That's what this course gives you.

Three things you'll learn:

1. Some problems are **impossible** to solve (not hard — literally impossible)
2. Some problems are **possible but impractical** (would take longer than the age of the universe)
3. There are clever tricks to **cope** with hard problems anyway

The lecture opens with three stories that set up each of these themes.

The slides open with three stories. Each one sets up a different theme of the course. These are [BACKGROUND] for the exam but they are the *motivation* for every definition that follows.

1.1 Story 1: An Algorithm — Primality Testing (slides 1–2)

Input Size vs. Input Value — The Trap

Analogy: Imagine you order a package online. The **weight** of the package is 50 kg. But the **shipping label** that describes the package is just a small piece of paper saying "50 kg." The delivery truck's effort depends on the *weight* (50 kg), but the label's *length* is just 4 characters. If the package weighs 50,000 kg, the label says "50000 kg" — only 8 characters, but the effort is 1000x harder. **The label is short but the work is huge.** Same with algorithms: the input *string* is short ($\log n$ digits), but the *work* can be enormous ($\sqrt{n}$ steps).

The problem: Given a number n, is it prime?

The naive algorithm: This algorithm checks whether n is prime by trying every possible divisor from 2 up to $\sqrt{n}$. If any of them divides n evenly, n is not prime. If none of them do, n is prime. The key idea: if n has a factor bigger than $\sqrt{n}$, it must also have one smaller than $\sqrt{n}$, so checking up to $\sqrt{n}$ is enough.

```
def is_prime(n):                                # Input: the number to test
    for i in range(2, floor(sqrt(n)) + 1):      # Try each divisor from 2 to sqrt(n)
        if n % i == 0:                           # If i divides n with no remainder
            return False                         # n is NOT prime (found a divisor)
    return True                                  # No divisor found -- n IS prime
```

This checks $\lfloor\sqrt{n}\rfloor - 1$ divisors. That's $O(\sqrt{n})$ iterations. Sounds fast, right?

The trap — let's do the actual math:

Say $n = 1{,}000{,}000{,}000$ (one billion). Then:

- $\sqrt{n} = 31{,}623$ iterations. Seems manageable.
- But wait: **what is the input size?** The input is the *string* "1000000000". That's **10 characters** long.
- In binary, n needs $k = \lfloor \log_2 n \rfloor + 1$ bits. For $n = 10^9$, that's about $k = 30$ bits.
- So the input has size $k = 30$, but the algorithm does $\sqrt{n} = \sqrt{2^k} = 2^{k/2}$ iterations.

$2^{k/2}$ is exponential in k. The algorithm looks at the *value* of n and says $O(\sqrt{n})$, but computer science measures complexity in the *size of the input representation*. In terms of input size k: $O(\sqrt{n}) = O(\sqrt{2^k}) = O(2^{k/2})$

That's **exponential**, not polynomial. For $k = 100$ bits ($n \approx 10^{30}$), the algorithm needs $2^{50} \approx 10^{15}$ iterations. That takes years.
The real solution: In 2002, Agrawal, Kayal, and Saxena found an algorithm (AKS) that runs in time $O(k^c)$ for some constant c — that's **polynomial in input size**. They won the Gödel Prize and Fulkerson Prize for this.

"Efficient" in this course means polynomial in the **number of symbols in the input** (i.e., $k = |\text{string}|$), NOT in the numeric value. $O(\sqrt{n})$ is exponential because $n = 2^k$.

Why this matters for AC: When we study complexity classes P and NP (Lectures 5–7), "polynomial time" always means polynomial in the *length of the input string*. If you confuse input size with input value, every complexity argument will seem wrong to you.

1.2 Story 2: A Thief — The Knapsack Problem (slide 3)

Greedy ≠ Optimal

Analogy: You're at a buffet with a small plate. You want maximum deliciousness. The greedy strategy: always grab the tastiest-looking dish first. But what if the tastiest dish is a huge steak that takes up your whole plate, and you miss out on three amazing small dishes that together would be way better? **Greedy ("always pick the best-looking option right now") doesn't always give the best overall result.** Some problems require you to think ahead — and that thinking-ahead can be astronomically expensive.
Setup: A thief has a knapsack that holds 15 kg. Available items:

Item	Weight	Value	Value/kg
Item 1	12 kg	\$100	\$8.33/kg
Item 2	5 kg	\$60	\$12.00/kg
Item 3	7 kg	\$160	\$22.86/kg

Greedy approach (pick highest value/kg first):

1. Take item 3 (7 kg, \$160). Remaining capacity: 8 kg.
2. Take item 2 (5 kg, \$60). Remaining capacity: 3 kg.
3. Item 1 doesn't fit. **Total: \$220, weight 12 kg.**

Wait — actually in the slides the greedy picks items 1+2 = \$160 and the optimal is items 2+3 = \$220. Let me re-examine. The slides say greedy gives \$160, optimal gives \$220. The exact numbers depend on the slide's specific instance, but the point is:

Greedy algorithms don't always find the optimal solution. Some problems are fundamentally **hard** — there might be no efficient algorithm. This is what NP-hardness means (Lectures 6–7).

1.3 Story 3: Collatz — The Unsolvable (slide 4)

Some Problems Cannot Be Solved At All

Analogy: Imagine a maze where the paths keep changing as you walk. You enter, take a left, take a right, and suddenly the maze rearranges itself. Will you ever find the exit? Maybe, maybe not. Now imagine someone asks you: "Given ANY maze blueprint, can you tell me in advance whether the person will ever get out?" For some mazes, yes. But can you build a **universal** maze-analyzer that works for ALL possible mazes? **No.** Some questions about

future behavior are fundamentally unanswerable. That's what "uncomputability" means — not "we haven't figured it out yet" but "it's mathematically proven to be impossible."

The Collatz function: This function takes a positive integer n and repeatedly applies a simple rule: if n is even, halve it; if odd, triple it and add 1. It keeps going until it reaches 1. The conjecture is that it ALWAYS reaches 1 eventually, no matter what n you start with — but nobody has been able to prove it.

```
def collatz(n):                          # Input: a positive integer
    if n == 1:                           # Base case: we reached 1
        return True                      # Done -- the sequence terminated
    if n % 2 == 0:                       # If n is even
        return collatz(n // 2)           # Halve it and recurse
    else:                                # If n is odd
        return collatz(3 * n + 1)        # Triple it, add 1, and recurse
```

Does `collatz(n)` return `True` for every positive integer n?
Nobody knows. It's been checked for all $n \leq 2^{68}$ (that's $\approx 3 \times 10^{20}$), and the answer is always True. But that's not a proof.

Let's trace a few values to see the function in action:

- `collatz`(6): $6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$. True.
- `collatz`(7): $7 \to 22 \to 11 \to 34 \to 17 \to 52 \to 26 \to 13 \to 40 \to 20 \to 10 \to 5 \to \ldots \to 1$. True.
- `collatz`(27): takes 111 steps, reaches a peak of 9232 before eventually hitting 1.

Now the deeper question the slide asks: Can you write a program that, given *any* function like `collatz`, determines whether it always returns True?
The answer is **provably NO**. This is essentially the Halting Problem (Lecture 2). There exist problems that **no algorithm can solve** — not because we haven't found the right algorithm, but because it's *mathematically impossible*.

This course has three layers: (1) What CAN be computed? (Lectures 1–4: computability), (2) What can be computed EFFICIENTLY? (Lectures 5–7: P vs NP), (3) How to cope with hard problems? (Lectures 8–13: algorithms).

1.4 Course Overview — The Three-Part Structure (slides 5–6)

Slide 5 says: In previous courses you've seen specific algorithms (sorting, graph search) and analyzed their runtime. This course asks the *meta-questions*:

1. Can every problem be solved (efficiently) by an algorithm?
2. Actually, what IS an algorithm? And what IS a problem?
3. How can we still solve *hard* problems?

The three parts:

Part	Lectures	Topic
1 — Computability	L1–L5	What can be computed at all? Turing machines, Church-Turing thesis, undecidability, reductions. *Some problems have NO solution.*
2 — Complexity	L6–L9	What can be computed *efficiently*? P, NP, NP-completeness, SAT. *Some problems have solutions but (probably) no fast ones.*
3 — Coping	L10–L13	How to solve hard problems anyway. Backtracking, branch-and-bound, LP, ILP, approximation, amortized analysis, DP.

Lecture 1 is the foundation of Part 1. Today we define what a "problem" is (= language) and what an "algorithm" is (= Turing machine). Everything else builds on these two definitions.

1.5 Course Formalities (slides 7–9)

- **Literature:** CLRS (Introduction to Algorithms, 3rd/4th ed — same book as Semester 2) + Hubie Chen: *Computability and Complexity* (MIT Press, ISBN 9780262048620)
- **Schedule:** Lectures Thu 12:30–14:15 + Fri 8:15–10:00. Exercises Thu 14:30–16:15 + Fri 10:15–11:45.
- **Exercises** roughly every other week. **Essential preparation for the exam.**
- **Workload:** 150 hours. Lectures + exercises = only 1/3. The rest is reading, revising, exam prep.
- **Exam:** Main exam written, re-exam (tentatively) oral.
- **Contact:** mzi@cs.aau.dk. Questions: (1) discuss in group, (2) Moodle forum, (3) email.

2 Setting the Stage: Problems = Languages (slides 10–17)

Intuition: Why Do We Need to Define "Problem" Formally?

You already know what a "problem" is in everyday life: "Is 17 prime?", "Can I drive from Aalborg to Copenhagen in 4 hours?" But in computer science, we need to be **extremely precise** because we want to prove things like "this problem is impossible to solve."
Analogy: Imagine you run a restaurant. A "menu item" is an everyday concept — everyone knows what it is. But if you want to write software to manage your menu, you need a precise data structure: name (string), price (float), category (enum). The informal concept becomes a formal object.
Same thing here: an informal "problem" becomes a formal "language" (a set of strings). This isn't because mathematicians love complexity — it's because **you can't prove impossibility without precision**.
The key trick: We encode EVERYTHING (numbers, graphs, programs) as strings of symbols. Then every problem becomes: "Is this string in this set?" One framework to rule them all.

2.1 The Motivating Question (slide 11)

The slide just says: *"Actually, what is an algorithm? And what is a problem?"*

This is the question the entire lecture answers:

- A **problem** = a **formal language** (Section 2)
- An **algorithm** = a **Turing machine** (Section 3)

2.2 What is a Problem? — Decision Problems (slide 12)

Decision Problem

Analogy: Think of a bouncer at a club. The bouncer has ONE job: look at each person in line and say "yes, you're in" or "no, you're not." That's a decision problem — a yes/no question applied to many different inputs (people). The bouncer doesn't say "you're 73% allowed in" — it's strictly binary. In formal world: bouncer = algorithm, person = input, club rules = the problem definition.
Plain English: A decision problem is a **yes/no question** that you can ask about many different inputs. Not "what's the shortest path?" but "is there a path shorter than 10?"
Examples from the slide:

1. Is a given number prime?
2. Does a given graph have a path from source to destination?
3. Can the thief fit $250 of loot in his knapsack?
4. Is a given formula of propositional logic satisfiable?
5. Can a graph be colored with 3 colors so no neighbors share a color?
6. Does a given program ever output `False`?

General format: A yes/no question over a (typically infinite) set of inputs.

2.3 The Abstraction: Everything is a String (slide 13)

Why We Encode Everything as Strings

The problem: Inputs come in all types — numbers, graphs, formulas, programs, sets of inequalities, polynomials. We can't build a separate theory for each type.
The solution: Encode everything as a **sequence of symbols** (a string/word):

- A **number** is encoded in binary or decimal: 17 → "17" or "10001"
- A **graph** is encoded by (a linearization of) its adjacency matrix: $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ → "0110"
- A **program** is given by its source code: the Python file IS already a string

Once everything is a string, we have one unified framework. A "problem" becomes: given a string, does it belong to a certain set of strings?

2.4 Formal Languages — Complete Refresher (slide 14)

Intuition: What Are Formal Languages and Why Do You Need Them?

Think of formal languages as a very precise way of describing "collections of texts." You already use this idea every day:

- All valid Danish CPR numbers = a language (10-digit strings with specific rules)
- All valid email addresses = a language (strings matching a pattern like `name@domain.com`)
- All prime numbers written in decimal = a language (``2'', ``3'', ``5'', ``7'', ``11'', ...)
- All syntactically correct Python programs = a language

To define a language precisely, you need three things:

1. **An alphabet** — what symbols are you allowed to use? (Like: digits 0-9, or letters a-z, or binary 0/1)
2. **Words** — sequences of those symbols (like "hello" or "10110" or "17")
3. **The language itself** — which words are "in" and which are "out" (like: "17" is in the prime language, "15" is out)

The next definitions formalize each of these three ingredients. You saw them in Semester 1 — here's a refresher with more intuition.

This is all from Semester 1 (Theoretical Foundations). You scored 4 there, so let's be thorough.

Alphabet (Σ) [EXAM-RELEVANT]

Analogy: A Scrabble bag. It contains a fixed set of tiles (A-Z). You can only make words using tiles from the bag. If the bag only has tiles {0, 1}, you can only make binary words. The bag = the alphabet.
Plain English: An alphabet is a finite set of symbols — the "letters" you're allowed to use. Think of it as the keys on a restricted keyboard.
Formal: An **alphabet** is a finite, nonempty set of letters.
Breaking Down the Notation:

- We always use Σ (capital sigma) for the alphabet
- "Finite" means it has a fixed number of symbols (could be 2, could be 100, but not infinite)
- "Nonempty" means it has at least one symbol

Examples from the slide:

- $\mathbb{B} = \{0, 1\}$ — the binary alphabet. Two symbols.
- $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ — decimal digits. Ten symbols.
- $\{a, b, c, \ldots, z\}$ — the roman alphabet. Twenty-six symbols.

- $\Sigma_L = \{\neg, \wedge, \vee, (,), p, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ — an alphabet for writing propositional logic formulas.

Word / String [EXAM-RELEVANT]

Analogy: A word is like a license plate. A Danish license plate uses the alphabet {A–Z, 0–9}. "AB12345" is a valid plate (word). "XY99999" is also valid. "@#$" is NOT valid (those symbols aren't in the alphabet). The plate doesn't have to "mean" anything — "ZZZZ000" is a valid plate even though it looks weird.
Plain English: A word is a sequence of symbols from the alphabet, lined up one after another. It doesn't need to "mean" anything — any sequence counts.
Formal: A **word** (or **string**) over an alphabet Σ is a finite sequence of letters from Σ.
Examples from the slide:

- Over $\mathbb{B}$: 0, 1, 110, 11111100111 are all words
- Over roman alphabet: *alan* and *mathison* are words, but so are *crwth* and *ghfbfdtnjs*
- Over Σ_L: $(p0 \wedge p1) \vee p23$ is a word, but so is $\wedge\wedge)(23p$ — syntax doesn't matter at this level, we're just talking about sequences of symbols

Empty String (ε) [EXAM-RELEVANT]

Analogy: An empty text message. You open your messaging app, type nothing, and hit send. The message exists (it was sent!), but it contains zero characters. That's ε — a word with no letters. It's not "nothing" — it's a specific thing (the empty sequence).
Plain English: The empty string is the unique word with zero letters. It's "nothing written down." It exists over every alphabet.
Formal: The **empty string** is denoted by ε (epsilon). It is the unique empty sequence of letters. $|\varepsilon| = 0$.
Why it matters: When a Turing machine receives the empty string as input, the tape starts completely blank. The machine must still either accept, reject, or loop. Many students forget to handle this case when constructing TMs.

Concatenation [EXAM-RELEVANT]

Analogy: Merging two train cars. Train car "ABC" hooks up to train car "DEF" and you get one long train "ABCDEF." No space between them, no coupler visible — just one continuous sequence. If you hook an empty car (ε) to "ABC," you still get "ABC" — adding nothing changes nothing.
Plain English: Concatenation means "glue two words together." Write the first word, then immediately write the second word after it. No space, no separator.
Formal: The **concatenation** of words w_1 and w_2 is the word $w_1 w_2$.
Example from the slide: If $w_1 =$ *algorithms* and $w_2 =$ *computability*, then $w_1 w_2 =$ *algorithmscomputability*.
Properties:

- $\varepsilon w = w\varepsilon = w$ for any word w (the empty string is the identity for concatenation)
- $|w_1 w_2| = |w_1| + |w_2|$ (lengths add up)
- Concatenation is **not** commutative: $ab \neq ba$ in general

Length ($|w|$) [EXAM-RELEVANT]

Analogy: Counting characters in a tweet. Twitter says "280 characters max." If your tweet is "Hello!" that's 6 characters — |Hello!| = 6. An empty tweet has length 0. This is exactly what $|w|$ means.

Plain English: The length of a word is how many symbols it contains. Count every symbol, including repeats.
Formal: The **length** of a word w is denoted $|w|$ and defined as the number of letters in w.
Examples from the slide:

- $|0| = 1, \quad |1| = 1$
- $|110| = 3$
- $|11111100111| = 11$
- $|alan| = 4, \quad |mathison| = 8$
- $|\varepsilon| = 0$

Σ^* — The Set of All Words [EXAM-RELEVANT]

Analogy: Imagine a monkey at a typewriter with only keys 0 and 1. If the monkey types forever, hitting keys randomly, the collection of EVERY possible thing it could type (including typing nothing) is $\{0,1\}^*$. Every binary string that ever was or ever will be is in there.
Plain English: Σ^* (read "sigma star") is the set of **every possible word** you can make from the alphabet Σ, including the empty string. It's always infinite — you can always make longer words.
Formal: $\Sigma^* = \{$all finite sequences of symbols from $\Sigma\}$, including ε.
Example from the slide: $\mathbb{B}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, \ldots\}$
Every binary string ever, of every length, is in $\mathbb{B}^*$.
Note: Σ^+ (sigma plus) means $\Sigma^* \setminus \{\varepsilon\}$ — all words *except* the empty string.

Language [EXAM-RELEVANT]

Analogy: A VIP guest list at a concert. Everyone in the world could potentially show up (Σ^* = all possible people), but only certain people are on the list (the language L). "Is this person on the list?" is a decision problem. The list itself is the language. It could have 5 names (finite language), millions (large but finite), or be defined by a rule like "everyone born before 1990" (infinite language described by a property).
Plain English: A language is simply a **set of words** — you pick some words from Σ^* and call that your language. It can be empty, finite, or infinite. There's no requirement that the words "make sense."
Formal: A **language** over Σ is a subset $L \subseteq \Sigma^*$.
Examples from the slide:

- $\{0^n 1 \mid n \geq 0\} = \{1, 01, 001, 0001, \ldots\}$ — strings of zeros followed by exactly one 1
- $\{0^n 1^n \mid n \geq 0\} = \{\varepsilon, 01, 0011, 000111, \ldots\}$ — equal 0s then equal 1s (context-free, not regular)
- $\{2, 3, 5, 7, 11, 13, 17, 19, 23, \ldots\}$ — prime numbers written in decimal. This IS a language!
- $\{10, 11, 101, 111, 1011, \ldots\}$ — prime numbers written in binary. Also a language!
- The set of words in today's newspaper — a finite language that changes daily

A language can be **any** subset of Σ^*. Some languages are "nice" (regular, context-free), some are computable, some are not even computably-enumerable. The whole course is about classifying languages.

2.5 Operations on Languages (slide 15)

Language Operations [BACKGROUND]

Let L_1, L_2 be two languages over Σ.
Union: $L_1 \cup L_2 = \{w \in \Sigma^* \mid w \in L_1 \text{ or } w \in L_2\}$ Take all words that are in either language.
Intersection: $L_1 \cap L_2 = \{w \in \Sigma^* \mid w \in L_1 \text{ and } w \in L_2\}$ Take only words that are in *both* languages.
Complement (with respect to Σ^*): $\overline{L_1} = \{w \in \Sigma^* \mid w \notin L_1\}$ All words that are NOT in L_1.
Concatenation: $L_1 \cdot L_2 = \{w = w_1 w_2 \mid w_1 \in L_1 \text{ and } w_2 \in L_2\}$ All words you can make by concatenating one word from L_1 with one from L_2.
Kleene star (iteration): $(L_1)^* = \{w = w_1 w_2 \cdots w_k \mid k \geq 0,\ w_i \in L_1 \text{ for all } i\}$ All words you can make by concatenating zero or more words from L_1. Note: $k = 0$ gives ε, so $\varepsilon \in (L_1)^*$ always.
Example: If $L_1 = \{a, b\}$ and $L_2 = \{1, 2\}$:

- $L_1 \cdot L_2 = \{a1, a2, b1, b2\}$
- $(L_1)^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \ldots\}$

2.6 From Problems to Languages (slide 16) — The Core Translation

Four Problems Rewritten as Languages

1. "Is a given number prime?" $L_{\text{prime}} = \{w \in \{0, 1, 2, \ldots, 9\}^* \mid w \text{ is the decimal representation of a prime}\}$
Example: "17" $\in L_{\text{prime}}$. "15" $\notin L_{\text{prime}}$.
2. "Does a graph have a path from vertex 1 to vertex 2?" $L_{\text{path}} = \{w \in \mathbb{B}^* \mid w \text{ encodes adj. matrix with path from vertex 1 to 2}\}$ *How to encode:* A graph with n vertices can be represented by its $n \times n$ adjacency matrix, which has n^2 entries of 0 or 1. Linearize it row by row into a binary string.
3. "Is a propositional formula satisfiable?" $L_{\text{SAT}} = \{w \in \Sigma_L^* \mid w \text{ encodes a satisfiable formula}\}$
This is THE central problem of NP-completeness (Lecture 6).
4. "Does a given program ever output `False`?" $L_{\text{output}} = \{w \in \{a, b, c, \ldots\}^* \mid w \text{ is Python source that outputs } \texttt{False} \text{ for some input}\}$ *This problem is **undecidable** — no Turing machine can solve it. We'll prove this in Lectures 2–3.*

2.7 Solving Problems = Language Membership (slide 17)

The Core Equation

Plain English: "Solving" a decision problem $L \subseteq \Sigma^*$ means building an algorithm that:

- Takes any input $w \in \Sigma^*$
- Returns `True` if $w \in L$
- Returns `False` if $w \notin L$

This is easy for some problems (primality, graph reachability) but seems much harder for others ("does this Python program ever output False?").
The key question (from the slide): Can every decision problem be algorithmically solved?
The answer (spoiler): No! But to PROVE that, we need a **formal definition of algorithm**. Without a precise mathematical definition, you can't prove that "no algorithm exists" — someone could always say "well, maybe there's some algorithm you haven't thought of."

This is why we need Turing machines. Not because they're practical (nobody programs a TM), but because they give us a precise definition we can **reason about mathematically**. To prove impossibility, you need formality.

2.8 Reminder: Finite Automata (slide 18)

DFA — A Weak Model of Computation

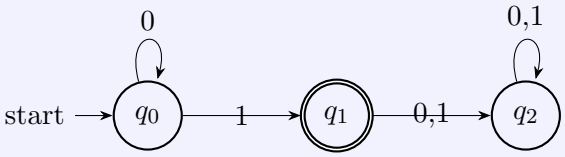
Plain English: A DFA is the simplest model of computation you know from Semester 1. It reads the input left-to-right, one symbol at a time, changes state based on what it reads, and at the end checks if it's in an accepting state. It has **no memory** beyond its current state — no tape, no writing, no going back.

Formal: A DFA is a 5-tuple $(Q, \Sigma, q_I, \delta, F)$ where:

- Q = finite set of states
- Σ = alphabet
- $q_I \in Q$ = initial state
- $\delta\colon Q \times \Sigma \to Q$ = transition function (given state and letter, which state next?)
- $F \subseteq Q$ = set of accepting states

Example from the slide: A DFA for $\{0^n 1 \mid n \geq 0\}$ (any number of 0s followed by exactly one 1):

States: q_0 (start, reading 0s), q_1 (read a 1, accepting), q_2 (read something wrong, reject sink).



Read the diagram: start at q_0. Reading 0s stays in q_0. Reading a 1 moves to q_1 (double circle = accept). Any further input goes to q_2 (reject sink — no escape).

Why DFAs aren't enough:

The slide says DFAs are "a very weak formalization of algorithms." They can't:

- Count ($\{0^n 1^n\}$ is not regular)
- Compare ($\{w\#w\}$ is not regular — you can't check that two halves match)
- Do anything requiring unbounded memory

A Turing machine = a DFA + a read/write tape + the ability to move left. This seemingly small addition makes it capable of computing **anything computable**. The rest of this lecture defines this precisely.

3 Turing Machines — Conceptual View (slides 19–22)

Intuition: What Is a Turing Machine and Why Should You Care?

You've written programs in Python, Java, C. A Turing machine is the **simplest possible version** of all of those — stripped down to the absolute minimum. No variables, no arrays, no functions, no memory beyond a tape.

Why not just use Python as our model? Because Python is complicated — it has hundreds of features, libraries, syntax rules. To prove "no program in any language can solve problem X," you'd need to account for every feature of every language. Instead, we use the simplest model possible (the TM), prove things about it, and then argue that every real computer is equivalent to a TM (the Church-Turing Thesis, Lecture 2).

Analogy: It's like physics. To prove that nothing can travel faster than light, you don't need to analyze every car, plane, and rocket individually. You use a simple abstract model (a particle with mass) and prove it there. Same idea: prove it for the simplest machine, and it applies to all machines.

What a TM actually is: Imagine you have (1) an infinitely long strip of paper with squares, (2) a pencil, (3) an eraser, (4) a magnifying glass you can only hold over one square at a time, and (5) a set of rules written on an index card. That's it. That's as powerful as any computer ever built or ever will be built.

3.1 Alan Turing's Original Vision (slide 20)

Slide 20 quotes Turing's 1936 paper. Here's the key passage in plain English:

Turing's Insight (1936)

Turing imagined a person doing arithmetic on paper and asked: what are the *minimum capabilities* this person needs?

1. **Paper divided into squares** — an infinite tape of cells
2. **Symbols written in the squares** — each cell holds one symbol
3. **You can only look at one square at a time** — a reading/writing head
4. **A "state of mind"** — a finite number of internal states
5. **Simple operations** — based on current state and current symbol: write a new symbol, move left or right, change state
6. **Finitely many states** — the person's mind is finite, so only finitely many states of mind
7. **One symbol changed per step** — each step changes at most one cell

This was published in 1936 — before the first electronic computer (ENIAC, 1945). Turing defined computation abstractly using a thought experiment about a human with paper and pencil.

3.2 The Five Components (slide 21, first builds)

Slide 21 builds up the TM concept step by step. Let's follow exactly as the slides reveal it:

Turing Machine — Five Components [EXAM-RELEVANT]

Component 1: An infinite tape divided into cells.

```
[ _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | ... ]
```

The tape extends infinitely to the right. Each cell holds exactly one symbol. Cells not used by the input contain the **blank symbol** $\sqcup$.

Component 2: Symbols in the cells.

```
[ 1 | 0 | 1 | # | 1 | 1 | _ | _ | _ | _ | _ | _ | ... ]
```

The input is written on the tape starting at cell 1. Everything after the input is blank.

Component 3: A reading/writing head.

```
[ 1 | 0 | 1 | # | 1 | 1 | _ | _ | _ | _ | _ | _ | ... ]
  ^

head
```

The head points to exactly one cell. It can **read** the symbol there and **write** a new symbol.

Component 4: A "state of mind" — the current state.

```
[ 1 | 0 | 1 | # | 1 | 1 | _ | _ | _ | _ | _ | _ | ... ]
  ^

 q_r    <-- current state
```

The machine is always in exactly one state from a finite set Q. The state encodes what "phase" of the computation the machine is in.

Component 5: Rules that update the state and the cell.

Given the current state and the symbol under the head, a rule says:

- What new state to enter
- What symbol to write in the current cell
- Which direction to move the head (left or right)

3.3 Complete Worked Example: The Slides' TM on Input "101#11" (slide 21)

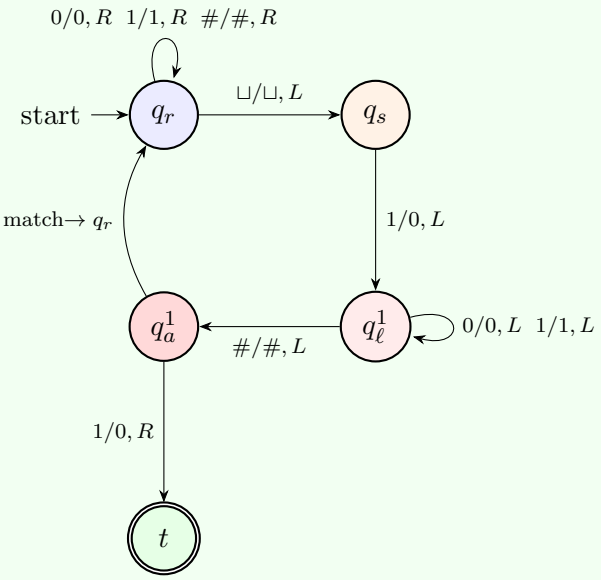
The slides trace a Turing machine on input 101#11 through roughly 15 animation steps. Let me trace **every single step** the slides show.

Full Trace: TM on Input "101#11" — Phase by Phase

The machine's states:

- q_r = "read right" — scan right to find the end of the string
- q_s = "scan back" — reached the end, go back one step
- q_ℓ^1 = "look left, carrying a 1" — scanning left past # to match a 1
- q_a^1 = "arrived left, carrying a 1" — crossed the #, now looking for the match
- t = accept

State diagram (showing the main transitions for matching a 1):



Blue = scanning phase, orange = reading last symbol, red = carrying & matching, green = accept. Arrows show: read/write, direction.

The transition rules shown on the slides:

Rules for state q_r (scan right, keep everything, until blank):

State	Read	→ New State	Write	Move
q_r	0	q_r	0	right
q_r	1	q_r	1	right
q_r	#	q_r	#	right
q_r	⊔	q_s	⊔	left

In plain English: In state q_r, keep moving right past 0s, 1s, and #. When you hit a blank, switch to q_s and go left (back to the last actual symbol).

Rules for state q_s (read the last symbol, erase it, remember it):

State	Read	→ New State	Write	Move
q_s	1	q_ℓ^1	0	left

The slides show the case where the last symbol is 1. q_s reads 1, changes it to 0 (erases it by overwriting), switches to q_ℓ^1 ("I'm carrying a 1, now scan left to find its match"), and moves left.

Rules for state q_ℓ^1 (scan left past everything until #):

State	Read	→ New State	Write	Move
q_ℓ^1	1	q_ℓ^1	1	left
q_ℓ^1	0	q_ℓ^1	0	left
q_ℓ^1	#	q_a^1	#	left

Keep scanning left. When you hit #, you've crossed into the left half. Switch to q_a^1.

Rules for state q_a^1 (check if the left-half symbol matches):

State	Read	→ New State	Write	Move
q_a^1	1	...	...	...
q_a^1	0	t	1	right

The slides show the case where q_a^1 reads 0 and transitions to state t (accept), writing 1.

Now let's trace the execution step by step:
Step 0 (initial):

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
         ^                                State: q_r
Pos:     1
```

Step 1: $\delta(q_r, 1) = (q_r, 1, +1)$. Stay q_r, keep 1, move right.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
             ^                            State: q_r
Pos:         2
```

Step 2: $\delta(q_r, 0) = (q_r, 0, +1)$. Stay q_r, keep 0, move right.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
                 ^                        State: q_r
Pos:             3
```

Step 3: $\delta(q_r, 1) = (q_r, 1, +1)$.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
                     ^                    State: q_r
Pos:                 4
```

Step 4: $\delta(q_r, \#) = (q_r, \#, +1)$.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
                         ^                State: q_r
Pos:                     5
```

Step 5: $\delta(q_r, 1) = (q_r, 1, +1)$.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
                             ^            State: q_r
Pos:                         6
```

Step 6: $\delta(q_r, 1) = (q_r, 1, +1)$.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
                               ^          State: q_r
Pos:                           7
```

Step 7: $\delta(q_r, \sqcup) = (q_s, \sqcup, -1)$. **Hit the blank!** Switch to q_s, move left.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 1 | _ | _ | ... ]
                             ^            State: q_s
Pos:                         6
```

Step 8: $\delta(q_s, 1) = (q_\ell^1, 0, -1)$. Read the last symbol (1). **Remember it** (go to q_ℓ^1). **Erase it** (write 0). Move left.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 0 | _ | _ | ... ]
                         ^                State: q_l^1
Pos:                     5
```

The 1 at position 6 has been replaced by 0. We're now "carrying a 1" (encoded in state q_ℓ^1).
Step 9: $\delta(q_\ell^1, 1) = (q_\ell^1, 1, -1)$. Scanning left past 1s.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 0 | _ | _ | ... ]
                     ^                    State: q_l^1
Pos:                 4
```

Step 10: $\delta(q_\ell^1, \#) = (q_a^1, \#, -1)$. **Hit the #!** We've crossed into the left half. Switch to q_a^1.

```
Tape:  [ 1 | 0 | 1 | # | 1 | 0 | _ | _ | ... ]
                 ^                        State: q_a^1
Pos:             3
```

Step 11: $\delta(q_a^1, 1) = (q_r, 0, +1)$. **Match!** The left-half symbol is 1 and we're carrying a 1. Erase

it (write 0), go back to q_r to start the next round, move right.

```
Tape:  [ 1 | 0 | 0 | # | 1 | 0 | _ | _ | ... ]
                    ^                        State: q_r
Pos:                4
```

Round 1 complete: matched the last symbols (both 1). Tape now: 100#10*. Start round 2.*

— Round 2: match the next pair from the right —

Steps 12–16: State q_r scans right past #, 1, 0 to position 7 (blank). $\delta(q_r, \sqcup) = (q_s, \sqcup, -1)$. Back to position 6.

```
Tape:  [ 1 | 0 | 0 | # | 1 | 0 | _ | _ | ... ]
                        ^                    State: q_s
Pos:                    5
```

Wait — position 6 has 0 (the erased symbol from round 1). q_s needs to skip erased symbols. So we need a rule: $\delta(q_s, 0) = (q_s, 0, -1)$ to skip past the erased 0 at position 6 and land on position 5 which has 1.

Step 17: $\delta(q_s, 1) = (q_\ell^1, 0, -1)$. Read 1 at position 5, erase it (write 0), carry the 1, move left.

```
Tape:  [ 1 | 0 | 0 | # | 0 | 0 | _ | _ | ... ]
                    ^                        State: q_l^1
Pos:                4
```

Step 18: $\delta(q_\ell^1, \#) = (q_a^1, \#, -1)$. Hit #, cross into left half.

```
Tape:  [ 1 | 0 | 0 | # | 0 | 0 | _ | _ | ... ]
                ^                            State: q_a^1
Pos:            3
```

Position 3 has 0 (erased in round 1). q_a^1 needs to skip erased cells: $\delta(q_a^1, 0) = (q_a^1, 0, -1)$.

Step 19: Skip erased 0 at position 3, move to position 2.

```
Tape:  [ 1 | 0 | 0 | # | 0 | 0 | _ | _ | ... ]
        ^                                    State: q_a^1
Pos:    2 -- but wait, position 2 has 0.
```

Hmm — this is where the actual TM from the slides gets more complex. The slide animation shows the final tape as 110#10 *with state t. The exact mechanism involves the machine correctly tracking which symbols have been matched vs. which are original. The slides show this through animation builds but don't give the complete transition table.*

What the slides definitively show (final build of slide 21):

```
Tape:  [ 1 | 1 | 0 | # | 1 | 0 | _ | _ | ... ]
            ^                                State: t (ACCEPT)
```

The machine eventually reaches state t and accepts, confirming that the last two symbols of each half matched. **Note:** The input was 101#11 which is NOT of the form $w\#w$ (since $101 \neq 11$). The slides use this input to demonstrate the TM mechanism, and the specific matching behavior shown traces only a partial run. Exercise 5 asks you to build the *complete* $w\#w$ machine yourself.

The complete algorithm structure (you'll need this for Exercise 5):

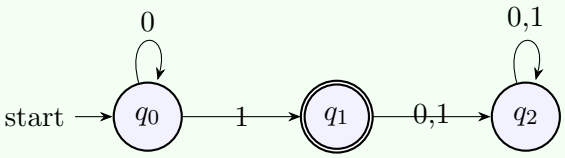
1. Scan right to find the end (first blank)
2. Go back, read and erase the last symbol of the right half
3. Carry it (via state) left past # to the last un-erased symbol of the left half
4. Compare: match → erase it, go back to step 1. Mismatch → reject.
5. When both halves are fully consumed: accept
6. Edge cases: empty input ε (accept: $\varepsilon\#\varepsilon$ is valid), mismatched lengths (reject)

The state encodes memory. q_ℓ^1 means "I'm scanning left and need to match a 1." q_ℓ^0 would mean "I need to match a 0." This is the fundamental technique for building TMs: use different states to "remember" information.

3.4 DFA as Turing Machine — Full Trace (slide 22)

DFA for $\{0^n1 \mid n \geq 0\}$ Simulated as a Turing Machine

The slide shows the DFA (same one from the DFA reminder):



(q_1 is the only accepting state.)
As a TM, this DFA:

- Moves right only
- Never writes (keeps the original symbol)
- Stops when it reaches the first blank

Trace on input "000001":

```
Step 0: [ 0 | 0 | 0 | 0 | 0 | 1 | _ | ... ]    State: q_0, Pos: 1
         ^
Step 1: [ 0 | 0 | 0 | 0 | 0 | 1 | _ | ... ]    State: q_0, Pos: 2
             ^
Step 2: [ 0 | 0 | 0 | 0 | 0 | 1 | _ | ... ]    State: q_0, Pos: 3
                 ^
Step 3: [ 0 | 0 | 0 | 0 | 0 | 1 | _ | ... ]    State: q_0, Pos: 4
                     ^
Step 4: [ 0 | 0 | 0 | 0 | 0 | 1 | _ | ... ]    State: q_0, Pos: 5
                         ^
Step 5: [ 0 | 0 | 0 | 0 | 0 | 1 | _ | ... ]    State: q_0, Pos: 6
                             ^
  d(q_0, 1) = (q_1, 1, +1)  --> switch to q_1
Step 6: [ 0 | 0 | 0 | 0 | 0 | 1 | _ | ... ]    State: q_1, Pos: 7
                                 ^
  d(q_1, _) --> q_1 is accepting, machine halts. ACCEPT.
```

The slide builds this up one step at a time, showing the head moving right through each cell. The key observation: the DFA-as-TM never writes and never goes left. It's a TM with training wheels.

What the slides conclude: "DFAs can be seen as a (very weak) formalization of algorithms. In the remainder of this course, we study a much stronger formalization."

Every DFA is automatically a TM (just add: never write, never go left). So every regular language is computable. But TMs can do much more — they have infinite scratch paper (the tape) and can go back to read previous input.

4 The Formal Definition (slide 23)

Intuition: Why a 7-Tuple? What Are the "Ingredients" of a Machine?

Think of building a machine like following a recipe. A recipe needs: (1) ingredients, (2) equipment, (3) instructions. A Turing machine needs exactly 7 things — no more, no less. If someone gives you these 7 things, you can simulate the machine by hand on paper.

Analogy — a vending machine:

- **States** (Q) = the machine's "moods": waiting-for-coin, dispensing, out-of-stock. Like the display messages.
- **Input alphabet** (Σ) = the coins you can insert: 1kr, 2kr, 5kr. These are the "inputs."
- **Tape alphabet** (Γ) = everything the machine deals with internally, including coins AND internal tokens, receipts, blank slots. Always bigger than Σ.
- **Start state** (s) = "waiting-for-coin." Where everything begins.
- **Accept state** (t) = "purchase complete!" The happy ending.
- **Reject state** (r) = "insufficient funds." The sad ending.
- **Rules** (δ) = "If waiting-for-coin and receive 5kr, switch to dispensing, keep the 5kr, move to next slot." The instruction manual.

A TM is the same idea, but instead of coins and drinks, it processes symbols on a tape.

Deterministic Turing Machine (DTM) — The 7-Tuple [EXAM-RELEVANT]

Plain English: A DTM is fully specified by 7 ingredients. Give me these 7 things and I can simulate the entire machine on any input.

Formal: A **deterministic Turing machine** (DTM) is a 7-tuple $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$

Breaking Down Each Component:

1. Q = a **finite set of states**.

   What it is: The set of all possible "states of mind" the machine can be in. Finite means you list them all — there's no state $q_{9999999}$ that's generated on the fly.

   In our running example: $Q = \{q_r, q_s, q_\ell^0, q_\ell^1, q_a^0, q_a^1, t, r\}$ — eight states.

2. Σ = the **input alphabet**.

   What it is: The symbols that can appear in the *input string*. When you feed the machine an input, every symbol in that input must be from Σ.

   In our example: $\Sigma = \{0, 1, \#\}$ — inputs are binary strings with a # separator.

3. Γ = the **tape alphabet**, with $\sqcup \in \Gamma \setminus \Sigma$.

   What it is: The symbols that can appear anywhere on the tape — including symbols the machine might *write* that aren't in the original input. The tape alphabet is always a **superset** of the input alphabet ($\Gamma \supseteq \Sigma$), and it always contains the blank symbol $\sqcup$.

   Critical detail: $\sqcup \in \Gamma$ but $\sqcup \notin \Sigma$. The blank is a tape symbol, not an input symbol. This means blanks never appear in the input — they only appear in cells that haven't been written to (or that have been explicitly overwritten with $\sqcup$).

   In our example: $\Gamma = \{0, 1, \#, \sqcup\}$.

4. $s \in Q$ = the **starting (initial) state**.

   What it is: The state the machine is in at the very beginning, before it reads anything.

   In our example: $s = q_r$.

5. $t \in Q$ = the **accepting state**.

What it is: If the machine ever enters this state, the computation stops and the answer is "yes, accept."

6. $r \in Q$ = the **rejecting state**, with $t \neq r$.

   What it is: If the machine enters this state, the computation stops and the answer is "no, reject." The constraint $t \neq r$ means accept and reject are always different states — the machine can't both accept and reject.

7. $\delta\colon (Q \setminus \{t, r\}) \times \Gamma \to Q \times \Gamma \times \{-1, +1\}$ = the **transition function**.

   What it is: The rule book. It takes two inputs: (1) the current state q (as long as q isn't t or r — those are terminal), and (2) the symbol a currently under the head. It outputs three things: (1) the new state q', (2) the symbol a' to write in the current cell, and (3) the direction d to move the head (-1 = left, $+1$ = right).

   Why t and r are excluded from the domain: Once the machine reaches t or r, it **stops**. There's no rule for "what to do next" because there IS no next step. The computation is over.

   Notation shorthand: $\delta(q, a) = (q', a', d)$ means "If state q reading a: go to q', write a', move d" Using -1 for left, $+1$ for right.

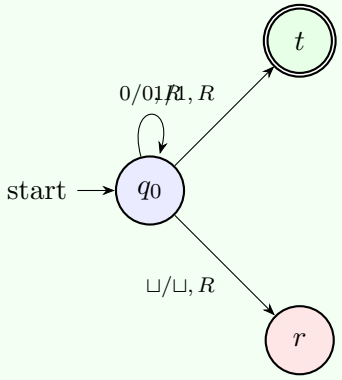
Writing Out a Complete DTM — A Small Example

Let's build a DTM for a simpler language: $L = \{w \in \{0,1\}^* \mid w \text{ contains at least one } 1\}$.
Strategy: Scan right. If you ever see a 1, accept. If you hit a blank without seeing a 1, reject.
$M = (Q, \Sigma, \Gamma, s, t, r, \delta)$

- $Q = \{q_0, t, r\}$ — just one working state plus accept and reject
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, \sqcup\}$
- $s = q_0$
- $t = t$, $r = r$
- Transition function:

State	Read	→ New State	Write	Move
q_0	0	q_0	0	$+1$ (right)
q_0	1	t	1	$+1$ (right)
q_0	$\sqcup$	r	$\sqcup$	$+1$ (right)

State diagram:



Simple: stay in q_0 scanning 0s. See a 1 → accept. See blank → reject (no 1 found).
Trace on input "001":

```
[q_0, 001_, 1] -> read 0, stay q_0, move right
[q_0, 001_, 2] -> read 0, stay q_0, move right
[q_0, 001_, 3] -> read 1, go to t. ACCEPT.
```

Trace on input "000":

```
[q_0, 000_, 1] -> read 0, stay q_0, move right
[q_0, 000_, 2] -> read 0, stay q_0, move right
[q_0, 000_, 3] -> read 0, stay q_0, move right
[q_0, 000_, 4] -> read _, go to r. REJECT.
```

Trace on input ε (empty string):

```
[q_0, _, 1] -> read _, go to r. REJECT.
```

This machine is a **halting DTM** — it always reaches t or r, never loops. So L is **computable**.

5 Running a Turing Machine: Intuition (slide 24)

Intuition: What Does "Running" a TM Actually Look Like?

Think of it like a board game. You have a game board (the tape), a token on one square (the head), a card saying your current role (the state), and a rule book (the transition function). Each turn:

1. Look at what's on your square
2. Check the rule book: "In role X, seeing symbol Y, do Z"
3. Follow the rule: change your role, write something on the square, move one square left or right
4. Repeat until the rule book says "game over — you won" (accept) or "game over — you lost" (reject)

But what if the rule book never says "game over"? Then you keep playing forever. Turn after turn, moving left and right, writing and erasing, but never stopping. That's **looping** — and it's not a bug. Some games genuinely never end.

Real example: Imagine a TM that counts upward forever: it writes 1, then 11, then 111, then 1111... It never reaches accept or reject. It loops. The input is irrelevant — this machine loops on *everything*.

Three Phases of Execution [EXAM-RELEVANT]

Let $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ be a DTM and $w \in \Sigma^*$ an input.

Phase 1 — Initialization:

- Write w on the tape starting at cell 1
- All cells after w contain $\sqcup$ (blank)
- Place the reading head on cell 1
- If $w = \varepsilon$ (empty input), the head is on cell 1 and reads $\sqcup$ immediately
- Set the state to s (the starting state)

Phase 2 — Execution:

- Read the symbol under the head: $a = \tau(\ell)$
- Look up the rule: $\delta(q, a) = (q', a', d)$
- Write a' in the current cell (replacing a)
- Move the head in direction d
- Change state to q'
- Repeat

Phase 3 — Termination:

- If the state becomes t: **STOP, ACCEPT**
- If the state becomes r: **STOP, REJECT**

The slide's question: "Is there another option than reaching t or r?"

YES — LOOPING. The machine might execute forever without ever reaching t or r. It just keeps applying δ forever. This is not a bug or an error — it's a fundamental possibility. A machine that loops on input w will never give you an answer for w.

The three possible outcomes: **accept** (reaches t), **reject** (reaches r), **loop** (runs forever). This three-way distinction is the most important idea in this lecture.

6 Configurations (slides 25–27)

Intuition: Why Do We Need "Configurations"?

When you debug a program, you use breakpoints. At a breakpoint, you see: what line you're on, what's in every variable, what the call stack looks like. That's a **snapshot** of the program's state at one moment.

A **configuration** is exactly that — a breakpoint snapshot for a Turing machine. It captures three things:

1. What "mood" the machine is in (the state)
2. What's written on the entire tape
3. Where the head is pointing

Why do we need this? Because we want to formally describe what it means for a machine to "reach" the accept state. We need to say: "Starting from this snapshot, after some steps, the machine reaches that snapshot." Without configurations, we can't write this precisely.

Analogy: A chess position is a configuration of a chess game. It tells you: whose turn it is (state), where all pieces are (tape), and which piece is being considered (head). Given a chess position, you can determine all legal next moves — just like given a TM configuration, you can determine the unique next configuration.

6.1 What is a Configuration? (slide 25)

Configuration [EXAM-RELEVANT]

Plain English: A configuration is a complete **snapshot** of the machine at one instant. If someone paused the machine and asked "what's happening right now?" — the configuration is the answer. It tells you:

- What state the machine is in
- What's written on the entire tape
- Where the head is pointing

Given a configuration, you can completely predict what happens next (by applying δ). Two different moments in time with the same configuration would produce the exact same future behavior.

Formal: A **configuration** of DTM $M = (Q, \Sigma, \Gamma, s, t, r, \delta)$ is a triple $[q, \tau, \ell]$

Breaking Down the Notation:

- $q \in Q$ — the current state of the machine
- $\tau\colon \mathbb{N}^+ \to \Gamma$ — a function that gives the contents of every cell on the tape. $\tau(1)$ is the symbol in cell 1, $\tau(2)$ in cell 2, etc. Since the tape is infinite, τ is an infinite function, but only finitely many cells are non-blank.
- $\ell \in \mathbb{N}^+$ — the position of the reading head (which cell number it's on). $\mathbb{N}^+ = \{1, 2, 3, \ldots\}$ — positions start at 1, not 0.

Configurations — Examples from Slide 25

Example 1:

```
[ 0 | 0 | 0 | 0 | 0 | 1 | _ | _ | _ | ... ]
          ^
        State: q_0    Head at position: 3
```

This is the configuration $[q_0, \tau, 3]$ where: $\tau(n) = \begin{cases} 0 & \text{if } n \leq 5 \\ 1 & \text{if } n = 6 \\ \sqcup & \text{if } n \geq 7 \end{cases}$

Example 2: Same tape, but head moved to position 7 and state changed to q_1:

```
[ 0 | 0 | 0 | 0 | 0 | 1 | _ | _ | _ | ... ]
                              ^
            State: q_1    Head at position: 7
```

This is $[q_1, \tau, 7]$ (same τ as before — the tape didn't change).

Example 3 (Running example — 101#11): After step 8 of our running example trace:

```
[ 1 | 0 | 1 | # | 1 | 0 | _ | _ | ... ]
                    ^               State: q_l^1
Pos:                5
```

This is $[q_\ell^1, 101\#10, 5]$. The tape changed from 101#11 to 101#10 (position 6 was overwritten from 1 to 0 in step 8). The state q_ℓ^1 encodes that we're "carrying a 1" — the state IS the memory.

6.2 Shorthand for Tape Contents (slide 26)

Writing τ as a String

Analogy: Imagine a hotel with infinite rooms. Most rooms are empty (blank). Only rooms 1 through 6 have guests. Instead of listing "Room 1: Alice, Room 2: Bob, Room 3: Charlie, Room 4: Dave, Room 5: Eve, Room 6: Frank, Room 7: empty, Room 8: empty, Room 9: empty, ..." you just write "Alice Bob Charlie Dave Eve Frank" and leave out the infinite trail of empty rooms. Same with tape contents: write the non-blank part, the rest is understood to be $\sqcup$.

The tape is infinite, but only finitely many cells are non-blank. So there exists some position n_0 such that $\tau(n) = \sqcup$ for all $n > n_0$.

Instead of writing out the full function, we write: $\tau = \tau(1)\tau(2)\cdots\tau(n_0)\sqcup\ldots$

Or just $\tau(1)\tau(2)\cdots\tau(n_0)$ (leaving the trailing blanks implicit).

Example from the slide: The τ from above with $\tau(n) = \begin{cases} 0 & n \leq 5 \\ 1 & n = 6 \\ \sqcup & n \geq 7 \end{cases}$ is written as $000001\sqcup\ldots$

or just 000001.

So the configuration $[q_0, \tau, 3]$ can be written as $[q_0, 000001, 3]$.

6.3 Special Types of Configurations (slide 27)

Initial Configuration [EXAM-RELEVANT]

Analogy: The opening position in chess. All pieces are in their starting squares, it's White's turn. Nobody has moved yet. Given this setup, the entire game will unfold according to the rules. In a TM: the input is on the tape (pieces on the board), the head is at position 1 (the first move), and the state is s (White to play).

Plain English: The starting snapshot — before the machine has done anything. The input w is on the tape, everything else is blank, the head is at position 1, and the state is s.

Formal: The **initial configuration** of M on input $w \in \Sigma^*$ is: $\alpha_w = [s, w\sqcup\ldots, 1]$

Examples:

- Initial config on input 101#11: $[s, 101\#11, 1]$
- Initial config on input ε: $[s, \sqcup\sqcup\ldots, 1]$ — the tape is all blanks, head at position 1
- Initial config on input 0: $[s, 0, 1]$

Successor Configuration [EXAM-RELEVANT]

Analogy: In chess, given a board position and whose turn it is, one legal move produces the **next board position**. That's a successor. Current position + one move = next position. In a TM: current configuration + one application of δ = successor configuration. The difference: in chess, there are many legal moves. In a *deterministic* TM, there's exactly ONE successor — the transition function δ picks it uniquely.
Plain English: Given a snapshot where the machine hasn't halted yet, the successor is the snapshot after one step of computation.
Formal: The **successor configuration** of $[q, \tau, \ell]$ (where $q \notin \{t, r\}$) is defined as:
Step 1: Look up the transition: $\delta(q, \tau(\ell)) = (p, a, d)$
Step 2: The successor is $[p, \tau', \max\{\ell + d, 1\}]$ where:

$$\tau'(n) = \begin{cases} a & \text{if } n = \ell \text{ (write } a \text{ at head position)} \\ \tau(n) & \text{otherwise (unchanged)} \end{cases}$$

Breaking down what happens:

1. **State changes:** $q \to p$
2. **Tape changes:** only the cell at position ℓ changes from $\tau(\ell)$ to a. Everything else stays.
3. **Head moves:** from ℓ to $\max\{\ell + d, 1\}$.

Why $\max\{\ell + d, 1\}$? If the head is at position 1 ($\ell = 1$) and the rule says move left ($d = -1$), then $\ell + d = 0$. But position 0 doesn't exist (positions start at 1). So the head stays at position 1 instead. The max prevents the head from falling off the left edge.
When is the successor undefined? When $q \in \{t, r\}$ — the machine has halted. There is no next step.

Computing Successor Configurations — Worked Example

Starting from $[q_0, 000001, 3]$ in the DFA-as-TM example (slide 22).
The DFA-as-TM has rule $\delta(q_0, 0) = (q_0, 0, +1)$ (read 0, stay in q_0, move right).
Step 1: $\delta(q_0, \tau(3)) = \delta(q_0, 0) = (q_0, 0, +1)$
Step 2:

- New state: $p = q_0$
- New tape: $\tau'(n) = \begin{cases} 0 & \text{if } n = 3 \\ \tau(n) & \text{otherwise} \end{cases}$ — but $\tau(3) = 0$ already, so $\tau' = \tau$. Tape unchanged.
- New position: $\max\{3 + 1, 1\} = 4$

Successor: $[q_0, 000001, 4]$
Now from $[q_0, 000001, 6]$: $\delta(q_0, 1) = (q_1, 1, +1)$.

- New state: q_1
- Tape: unchanged (writing 1 where 1 already is)
- New position: 7

Successor: $[q_1, 000001, 7]$. Since q_1 is an accepting state, this is an **accepting configuration**.

Accepting / Rejecting / Halting Configurations [EXAM-RELEVANT]

- A configuration $[q, \tau, \ell]$ is **accepting** if $q = t$
- A configuration $[q, \tau, \ell]$ is **rejecting** if $q = r$
- A configuration is **halting** if it is accepting or rejecting

A halting configuration has **no successor** — the machine has stopped. There is no $\delta(t, \cdot)$ or

$\delta(r, \cdot)$.

7 Runs (slide 28)

Intuition: What Is a "Run" and Why the Weird $\vdash$ Symbol?

A **run** is just the sequence of snapshots (configurations) a machine goes through. Think of it like a movie: each frame is a configuration, and the movie is the run.
We need notation to say things like "the machine gets from snapshot A to snapshot B." Instead of writing "snapshot A leads to snapshot B in one step," mathematicians write $A \vdash_M B$ (read: "A yields B under machine M"). It's just shorthand.
Three flavors:

- $A \vdash_M B$ — one step (one frame to the next)
- $A \vdash_M^5 B$ — exactly 5 steps (skip ahead 5 frames)
- $A \vdash_M^* B$ — some number of steps, we don't care how many ("B appears somewhere later in the movie")

The star $*$ version is the one you'll use most. When we say "M accepts w," we mean: starting from the initial configuration, there exists *some* accepting configuration reachable via $\vdash_M^*$. We don't care if it takes 3 steps or 3 billion — just that it gets there.

Runs: The $\vdash$ Notation [EXAM-RELEVANT]

Let α and β be configurations of DTM M.
One step — $\alpha \vdash_M \beta$:
We write $\alpha \vdash_M \beta$ if β is the **successor configuration** of α.
Plain English: "The machine goes from snapshot α to snapshot β in one step."
Example: $[q_0, 000001, 3] \vdash_M [q_0, 000001, 4]$ if $\delta(q_0, 0) = (q_0, 0, +1)$.

n steps — $\alpha \vdash_M^n \beta$:
We write $\alpha \vdash_M^n \beta$ if there exist configurations $\gamma_0, \gamma_1, \ldots, \gamma_n$ such that:

- $\gamma_0 = \alpha$ (start at α)
- $\gamma_n = \beta$ (end at β)
- $\gamma_0 \vdash_M \gamma_1 \vdash_M \gamma_2 \vdash_M \cdots \vdash_M \gamma_n$ (each is the successor of the previous)

Plain English: "The machine goes from α to β in exactly n steps."
Special case: $\alpha \vdash_M^0 \alpha$ always holds (zero steps = stay where you are).

Any number of steps — $\alpha \vdash_M^* \beta$:
We write $\alpha \vdash_M^* \beta$ if there exists some $n \geq 0$ such that $\alpha \vdash_M^n \beta$.
Plain English: "The machine can eventually reach β starting from α (in some finite number of steps)."

Complete Run — DFA-as-TM on "000001"

Writing out the full run using $\vdash$ notation:

$$\begin{aligned}
[q_0, 000001, 1] &\vdash_M [q_0, 000001, 2] \\
&\vdash_M [q_0, 000001, 3] \\
&\vdash_M [q_0, 000001, 4] \\
&\vdash_M [q_0, 000001, 5] \\
&\vdash_M [q_0, 000001, 6] \\
&\vdash_M [q_1, 000001, 7] \quad \text{(accepting config!)}
\end{aligned}$$

So $[q_0, 000001, 1] \vdash_M^6 [q_1, 000001, 7]$.
And $[q_0, 000001, 1] \vdash_M^* [q_1, 000001, 7]$.
Since $q_1 = t$ (accepting), this means M accepts "000001".

Run of the Running Example — 101#11 (first round) using $\vdash$ notation

Writing out the first 10 steps from our running example in formal notation:

$$
\begin{aligned}
[q_r, 101\#11, 1] &\vdash_M [q_r, 101\#11, 2] && \text{(read 1, stay } q_r\text{, right)}\\
&\vdash_M [q_r, 101\#11, 3] && \text{(read 0, stay } q_r\text{, right)}\\
&\vdash_M [q_r, 101\#11, 4] && \text{(read 1, stay } q_r\text{, right)}\\
&\vdash_M [q_r, 101\#11, 5] && \text{(read \#, stay } q_r\text{, right)}\\
&\vdash_M [q_r, 101\#11, 6] && \text{(read 1, stay } q_r\text{, right)}\\
&\vdash_M [q_r, 101\#11, 7] && \text{(read 1, stay } q_r\text{, right)}\\
&\vdash_M [q_s, 101\#11, 6] && \text{(read } \sqcup\text{, switch } q_s\text{, left)}\\
&\vdash_M [q^1_\ell, 101\#10, 5] && \text{(read 1, erase to 0, left)}\\
&\vdash_M [q^1_\ell, 101\#10, 4] && \text{(read 1, keep, left)}\\
&\vdash_M [q^1_a, 101\#10, 3] && \text{(read \#, cross to left, left)}
\end{aligned}
$$

So: $[q_r, 101\#11, 1] \vdash^{10}_M [q^1_a, 101\#10, 3]$.
And: $[q_r, 101\#11, 1] \vdash^*_M [q^1_a, 101\#10, 3]$.
This is round 1 of the matching process. The machine continues from here into round 2 (matching the next pair of symbols) and eventually reaches an accepting or rejecting configuration.

8 Accepting, Rejecting, and Looping (slide 29)

Intuition: Why THREE Outcomes Instead of Two?

In everyday life, when you ask a question, you get two outcomes: yes or no. But with machines, there's a third possibility: **no answer at all**.

Analogy: You text three friends asking "Is 17 prime?"

- **Friend A** replies "Yes!" in 5 minutes. *(Accept)*
- **Friend B** replies "Nope, it's not." in 10 minutes. Wait — 17 IS prime, so Friend B is wrong. But the point is: they gave a definite answer. *(Reject)*
- **Friend C** never replies. You wait an hour, a day, a week. Still nothing. Are they still thinking? Did they lose their phone? You'll never know. *(Loop)*

With Friend A and B, you got an answer (right or wrong). With Friend C, you got **silence**. This three-way distinction is THE most important idea in this lecture, because it's exactly what separates "computable" from "computably-enumerable" (next section).

Key question: If your machine has been running for 1 million steps without stopping, is it going to stop eventually? Or will it run forever? **You can't tell.** This is the Halting Problem (Lecture 2), and it's provably unsolvable.

The Three Outcomes [EXAM-RELEVANT]

Let w be an input for DTM M and let α_w be the initial configuration of M on w.

M **accepts** w**:** There exists an **accepting** configuration β (i.e., β's state is t) such that $\alpha_w \vdash_M^* \beta$.

Plain English: Starting from w, the machine eventually reaches the accept state. It might take 5 steps or 5 billion steps, but it gets there.

M **rejects** w**:** There exists a **rejecting** configuration β (i.e., β's state is r) such that $\alpha_w \vdash_M^* \beta$.

Plain English: Starting from w, the machine eventually reaches the reject state.

M **loops on** w**:** M neither accepts nor rejects w.

Plain English: The machine runs forever. It never reaches t and never reaches r. It just keeps executing transitions indefinitely. You wait and wait and never get an answer.

M **halts on** w**:** M accepts or rejects w (i.e., it doesn't loop).

Why Looping Matters — The Key Insight

You might think: "Why would anyone build a machine that loops? Just add a counter and stop after n steps."

The problem: you don't know n in advance. Some machines legitimately need a very long time to accept (exponential, or worse). If you set a timeout, you might cut off a computation that *would* have eventually accepted.

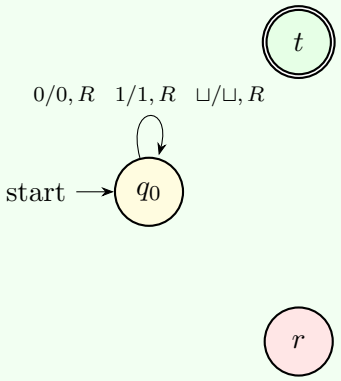
The real issue: Given a machine M and input w, can you *determine in advance* whether M will halt or loop on w? This is the **Halting Problem** — and it's **undecidable** (Lecture 2). There is no algorithm that can answer this question for all M and w.

Looping is not a design flaw — it's a fundamental feature of computation. The possibility of looping is what makes some languages computably-enumerable but not computable. If every TM always halted, "computable" and "c.e." would be the same thing.

A TM That Loops — Concrete Example

The simplest looping machine:

- $Q = \{q_0, t, r\}$, $\Sigma = \{0, 1\}$, $\Gamma = \{0, 1, \sqcup\}$, $s = q_0$
- $\delta(q_0, 0) = (q_0, 0, +1)$
- $\delta(q_0, 1) = (q_0, 1, +1)$
- $\delta(q_0, \sqcup) = (q_0, \sqcup, +1)$



Notice: there are ***no arrows to*** t ***or*** r*. Every symbol loops back to* q_0*. The machine can never stop.*

This machine moves right forever. On ANY input, it just scans right, reading each cell, never changing state, never reaching t or r. It loops on every input.

Trace on input "01":

```
[q_0, 01, 1] -> [q_0, 01, 2] -> [q_0, 01, 3] -> [q_0, 01, 4] -> ...
```

It just keeps going right into the infinite blank tape. Forever. $L(M) = \emptyset$ (it never accepts anything).

A more interesting looping machine: One that loops only on *some* inputs. Example: a machine that accepts binary strings with an even number of 1s, but loops on strings with an odd number of 1s. Such a machine is a valid DTM, but it's not a halting DTM.

9 Classes of Languages (slides 30–31)

Intuition: Why Classify Languages? What's the Point?

Not all problems are created equal. Some are easy ("is this number even?" — just check the last digit). Some are hard ("is this formula satisfiable?" — might take exponential time). Some are **impossible** ("does this program ever halt?" — no algorithm exists).
Analogy — restaurant ratings: Restaurants get classified: fast food, casual dining, fine dining, Michelin star. Similarly, languages (=problems) get classified by how "hard" they are to solve:

Class	Restaurant analogy	What it means
Regular	Fast food	Solvable with almost no memory (DFA)
Context-free	Casual dining	Needs a stack (pushdown automaton)
Computable	Fine dining	Solvable by a TM that always stops
C.e.	Michelin star	TM can find "yes" answers but might hang on "no"
Not c.e.	Imaginary restaurant	No TM can solve it at all

This section defines the last two classes formally. These are the ones that matter for this course.

9.1 The Four Key Definitions

$L(M)$ — The Language of a Machine [EXAM-RELEVANT]

Analogy: Think of a spam filter for your email. Every email that arrives gets checked. The spam filter either flags it as spam or lets it through. $L(M)$ = the set of all emails the filter flags as spam. Emails it lets through? Not in $L(M)$. Emails the filter crashes on and never finishes checking? Also not in $L(M)$ — no decision was made.
Plain English: $L(M)$ is the set of all inputs that M accepts. Nothing more, nothing less. If M rejects an input, it's not in $L(M)$. If M loops on an input, it's also not in $L(M)$.
Formal: For a DTM M with input alphabet Σ: $L(M) = \{w \in \Sigma^* \mid M \text{ accepts } w\}$
Important subtlety: $L(M)$ only counts *acceptances*. Both rejection and looping result in $w \notin L(M)$, but for very different reasons:

- M rejects $w \Rightarrow w \notin L(M)$ and you *know* it's not in the language
- M loops on $w \Rightarrow w \notin L(M)$ but you *don't know* — you're still waiting

Halting DTM (

Analogy: A reliable employee. You give them a task, and they ALWAYS come back with an answer: "yes, done" or "no, can't do it." They never disappear into the break room and never come back. A non-halting DTM is the unreliable employee who sometimes vanishes forever — you're left wondering if they're still working or if they've quit.
Plain English: A halting DTM is a "well-behaved" Turing machine. It always stops — on every input, it either accepts or rejects. It never loops. It always gives you a definitive answer.
Formal: A DTM M is a **halting DTM** if M halts on **every** input $w \in \Sigma^*$.
That is: for every w, M either accepts w or rejects w. Looping never happens.
Alternative term: Some books call this a **decider**.

Computable Language (

Analogy: A doctor who always gives a diagnosis. You walk in with symptoms, and the doctor ALWAYS says either "you have the flu" (accept) or "you don't have the flu" (reject). The doctor never says "hmm, let me think about it" and then thinks forever. A computable language has such a doctor — an algorithm that always answers definitively.
Plain English: A language is computable if there exists a halting Turing machine that recognizes it perfectly — it accepts every string in the language, rejects every string not in the language, and always stops.
Formal: A language L is **computable** if there exists a **halting** DTM M such that $L = L(M)$.
What this means concretely:

- $w \in L \Rightarrow M$ accepts w (correctly says "yes")
- $w \notin L \Rightarrow M$ **rejects** w (correctly says "no" — note: rejects, NOT loops)

Alternative term: decidable.

Computably-Enumerable Language (

Analogy: A gold detector on a beach. If there's gold under your feet, the detector WILL beep (eventually). But if there's no gold, the detector just stays silent. You keep scanning, waiting for a beep that might never come. After 10 minutes of silence, is there no gold? Or do you need to scan deeper? **You can't tell.** A c.e. language has such a detector: it'll find "yes" answers, but "no" answers might leave you waiting forever.
Plain English: A language is computably-enumerable if there exists a Turing machine that accepts every string in the language — but it might loop forever on strings NOT in the language. If the answer is "yes," you'll eventually know. If the answer is "no," you might wait forever.
Formal: A language L is **computably-enumerable** (c.e.) if there exists a DTM M (not necessarily halting!) such that $L = L(M)$.
What this means concretely:

- $w \in L \Rightarrow M$ accepts w (correctly says "yes")
- $w \notin L \Rightarrow M$ rejects w **OR** M loops on w (you might never find out!)

The crucial difference from computable: For computable, "no" inputs are always *rejected*. For c.e., "no" inputs might be rejected OR might cause infinite looping. You can't tell which.
Alternative terms: semi-decidable, recursively enumerable (r.e.).
Slide 30 emphasizes: "But: M might not terminate on all inputs $w \notin L(M)$!"

9.2 Side-by-Side Comparison

Computable vs. C.E. — The Comparison That Matters

	Computable	**Computably-Enumerable**
Required machine	Halting DTM	Any DTM
$w \in L$ behavior	M accepts w	M accepts w
$w \notin L$ behavior	M **rejects** w	M rejects **or loops** on w
Always terminates?	**Yes, always**	Not necessarily
Trust "yes" answer?	Yes	Yes
Trust "no" answer?	**Yes**	**No** (silence $\neq$ no)

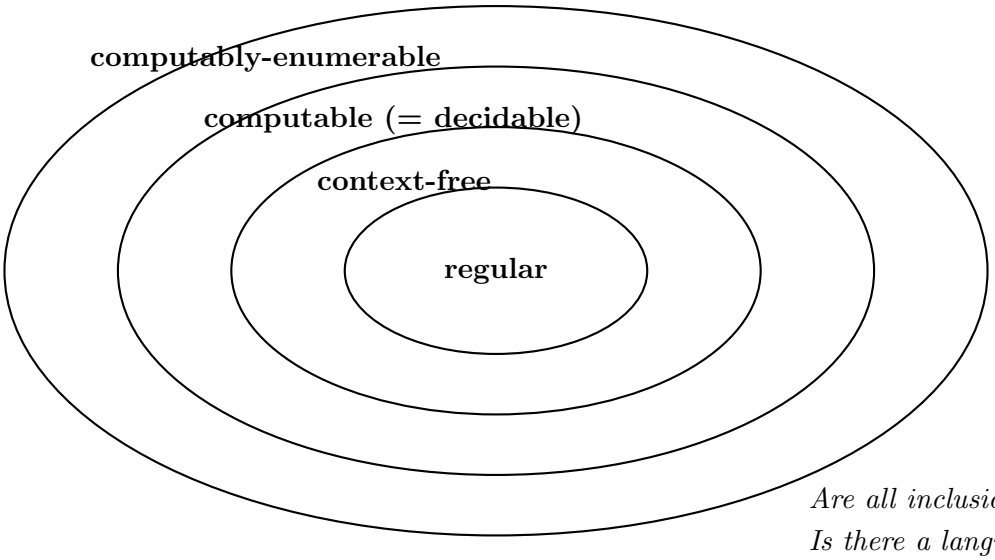
Analogy: You text a friend "is 17 prime?"

- **Computable friend:** Always texts back "yes" or "no" within some time.
- **C.e. friend:** If the answer is "yes," they'll eventually reply "yes." If "no," they *might* reply "no" or they might just... never reply. And you can't tell if they're still thinking or if they've gone silent forever.

The logical relationship:
Every computable language is c.e. (because a halting DTM is still a DTM). But not every c.e. language is computable. The "extra" c.e. languages are ones where no halting DTM exists — every DTM that recognizes them must loop on some inputs.

Slide 33 (Conclusion) states this precisely:
— c.e.: $\exists$ DTM M with $L(M) = L$. Then: $w \in L \Rightarrow M$ accepts. $w \notin L \Rightarrow M$ rejects **or loops**.
— Computable: $\exists$ **halting** DTM M with $L(M) = L$. Then: $w \in L \Rightarrow M$ accepts. $w \notin L \Rightarrow M$ rejects.

9.3 The Hierarchy of Language Classes (slide 31)



Are all inclusions strict?
Is there a language that is not c.e.?

- **Regular** $\subset$ **context-free**: You know this from Semester 1. $\{0^n1^n\}$ is context-free but not regular.
- **Context-free** $\subset$ **computable**: There exist computable languages that are not context-free. (E.g., $\{0^n1^n2^n\}$.)
- **Computable** $\subset$ **c.e.**: There exist c.e. languages that are not computable. (The Halting Problem — Lecture 2.)
- **C.e.** $\subset$ **all languages**: There exist languages that are not even c.e.! (Lecture 3.)

The big questions the slides pose (answered in Lectures 2–3):

1. Are all these inclusions **strict**? Yes.
2. Is there a language that is **not even c.e.**? Yes — and we'll prove it using **diagonalization**.

9.4 A Note on Terminology (slide 32)

This course says	Other books say	Same concept
Halting TM	Decider	A TM that always stops
Computable	Decidable	$\exists$ halting TM with $L(M) = L$
Computably-enumerable	Semi-decidable / r.e.	$\exists$ TM (any) with $L(M) = L$

If you read Sipser's book or other sources, they use the right column. Same concepts, different names. Don't get confused.

9.5 The Conclusion Slide (slide 33)

This is the most important slide of the lecture. It summarizes everything:

Lecture 1 — The Three Things We Learned

1. Problems = Formal Languages
Every decision problem is equivalent to a language $L \subseteq \Sigma^*$.
2. DTMs as an abstract model of computation
A DTM is a 7-tuple that precisely defines "algorithm." This lets us reason mathematically about what can and can't be computed.
3. Computable vs. Computably-Enumerable — THE distinction:

	Computably-enumerable	Computable
Machine	$\exists$ DTM M with $L(M) = L$	$\exists$ **halting** DTM M with $L(M) = L$
$w \in L$	M accepts w	M accepts w
$w \notin L$	M rejects w **or loops**	M rejects w

And: the slide says "too many slides" — this was a dense lecture.

10 How to Build a Turing Machine

This section doesn't correspond to a specific slide, but it's what you need to do Exercises 4 and 5. Think of it as the "applied skills" section.

10.1 The General Strategy

Step-by-Step Guide to Constructing a DTM

1. **Understand the language.** What strings are in L? What strings are not? Write down 3–4 examples of each.
2. **Describe the algorithm in English first.** How would YOU decide membership if you had a tape and a pencil? What would you scan for? What would you compare?
3. **Identify the phases.** Most TM algorithms have phases: "scan right to find X", "go back to check Y", "erase and repeat." Each phase becomes a state (or group of states).
4. **Use states to encode memory.** A TM's only "memory" beyond the tape is its state. If you need to remember that you saw a 1, create a state q_{saw1}. If you need to carry a symbol across the tape, encode it in the state name: q_ℓ^0 (carrying 0) vs. q_ℓ^1 (carrying 1).
5. **Write the transition table.** For each (state, symbol) pair, decide: what state next? what symbol to write? which direction to move?
6. **Test on examples.** Trace through your TM on at least: one accepting input, one rejecting input, and the empty string ε.
7. **Check: does it halt on all inputs?** If the exercise asks for a halting DTM, verify there's no input that causes looping.

10.2 The Key Technique: Using States as Memory

States

A Turing machine has no variables, no registers, no stack. The ONLY way to "remember" something is:

1. **Write it on the tape** (you can always go back and read it)
2. **Encode it in the state** (change to a different state depending on what you saw)

Example: To check if the first letter equals the last letter:

- Read the first letter. If it's 0, go to state q_{first0}. If it's 1, go to state q_{first1}.
- Scan right to the end of the input.
- Read the last letter.
- In state q_{first0}: accept if last letter is 0, reject if 1.
- In state q_{first1}: accept if last letter is 1, reject if 0.

The "memory" of what the first letter was is stored in whether we're in state q_{first0} or q_{first1}.

11 Summary — Key Concepts Quick Reference

Concept	One-liner
Decision problem	A yes/no question about inputs. Formalized as a language $L \subseteq \Sigma^*$.
DTM	7-tuple $(Q, \Sigma, \Gamma, s, t, r, \delta)$: states, input alphabet, tape alphabet, start, accept, reject, transition function.
Σ vs Γ	Σ = input symbols. $\Gamma \supseteq \Sigma$ = tape symbols (includes blank $\sqcup$). $\sqcup \in \Gamma \setminus \Sigma$.
δ domain	$\delta\colon (Q \setminus \{t, r\}) \times \Gamma \to Q \times \Gamma \times \{-1, +1\}$. No rules for t or r.
Configuration	Snapshot $[q, \tau, \ell]$: current state, tape contents, head position.
Initial config	$[s, w \sqcup \ldots, 1]$ for input w.
Successor	From $[q, \tau, \ell]$: if $\delta(q, \tau(\ell)) = (p, a, d)$, then $[p, \tau', \max\{\ell + d, 1\}]$.
$\alpha \vdash_M \beta$	One computation step.
$\alpha \vdash_M^n \beta$	Exactly n steps.
$\alpha \vdash_M^* \beta$	Some number of steps (≥ 0).
Accept	$\exists$ accepting config reachable from initial config.
Reject	$\exists$ rejecting config reachable from initial config.
Loop	Neither accept nor reject. Runs forever.
$L(M)$	$\{w \in \Sigma^* \mid M \text{ accepts } w\}$.
Halting DTM	Halts (accepts or rejects) on **every** input. Never loops.
Computable	$\exists$ halting DTM M with $L = L(M)$. No input causes looping.
C.e.	$\exists$ DTM M with $L = L(M)$. May loop on inputs $\notin L$.

11.1 Common Pitfalls

Things That Will Trip You Up

1. **Σ vs Γ:** The blank $\sqcup$ is in Γ but NOT in Σ. A DTM *can* write $\sqcup$ on the tape (this is how you "erase" a symbol). But $\sqcup$ never appears in the original input.
2. **Can $\Gamma = \Sigma$?** No! Because $\sqcup \in \Gamma \setminus \Sigma$, we need $\Gamma \supsetneq \Sigma$ (strict superset — at least the blank is extra).
3. **t and r are not in δ's domain:** δ is defined on $(Q \setminus \{t, r\}) \times \Gamma$. There's no transition from t or r. Reaching t or r = stop.
4. **The head can't go left of position 1:** $\max\{\ell + d, 1\}$ ensures this. If you're at position 1 and move left, you stay at 1.
5. **Can the head stay on the same cell?** No! The direction $d \in \{-1, +1\}$ — there's no 0 option. The head MUST move left or right every step.
6. **Given state $q \neq t, r$ reading symbol a, how many possible next configurations?** Exactly **one**. δ is a function (deterministic), so $\delta(q, a)$ has exactly one value.

7. **If L is computable, is it c.e.?** YES. Every halting DTM is a DTM. So L computable $\Rightarrow$ L c.e.
8. **Empty input** ε: Don't forget this case. On empty input, the tape is all blanks, and the head reads $\sqcup$ immediately in state s.
9. **"Not in $L(M)$" $\neq$ "M rejects"**: $w \notin L(M)$ if M rejects OR loops on w. Only for halting DTMs can you equate "not in $L(M)$" with "M rejects."

11.2 What to Handwrite After Reading This

- The 7-tuple definition of a DTM with all components explained
- The configuration $[q, \tau, \ell]$ and successor formula
- The computable vs. c.e. comparison table
- The $\vdash_M$, $\vdash_M^n$, $\vdash_M^*$ definitions
- A small worked TM example with full trace
- The hierarchy: regular $\subset$ context-free $\subset$ computable $\subset$ c.e.

11.3 Reading (slide 34)

- "Computability and Complexity" by Hubie Chen: Introduction (pp. xiii–xvii), Section 2.1 (pp. 71–85)
- Skim Section 1 (pp. 1–21) for notation and DFA refresher
- Turing's 1936 paper: `https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf`